



Smart Contract Security Audit Report



Table Of Contents

1 Executive Summary	_____
2 Audit Methodology	_____
3 Project Overview	_____
3.1 Project Introduction	_____
3.2 Vulnerability Information	_____
4 Code Overview	_____
4.1 Contracts Description	_____
4.2 Visibility Description	_____
4.3 Vulnerability Summary	_____
5 Audit Result	_____
6 Statement	_____

1 Executive Summary

On 2026.07.29, the SlowMist security team received the edgeX team's security audit application for EdgexBridge, developed the audit plan according to the agreement of both parties and the characteristics of the project, and finally issued the security audit report.

The SlowMist security team adopts the strategy of "white box lead, black, grey box assists" to conduct a complete security test on the project in the way closest to the real attack.

The test method information:

Test method	Description
Black box testing	Conduct security tests from an attacker's perspective externally.
Grey box testing	Conduct security testing on code modules through the scripting tool, observing the internal running status, mining weaknesses.
White box testing	Based on the open source code, non-open source code, to detect whether there are vulnerabilities in programs such as nodes, SDK, etc.

The vulnerability severity level information:

Level	Description
Critical	Critical severity vulnerabilities will have a significant impact on the security of the DeFi project, and it is strongly recommended to fix the critical vulnerabilities.
High	High severity vulnerabilities will affect the normal operation of the DeFi project. It is strongly recommended to fix high-risk vulnerabilities.
Medium	Medium severity vulnerability will affect the operation of the DeFi project. It is recommended to fix medium-risk vulnerabilities.
Low	Low severity vulnerabilities may affect the operation of the DeFi project in certain scenarios. It is suggested that the project team should evaluate and consider whether these vulnerabilities need to be fixed.
Weakness	There are safety risks theoretically, but it is extremely difficult to reproduce in engineering.

Level	Description
Suggestion	There are better practices for coding or architecture.

2 Audit Methodology

The security audit process of SlowMist security team for smart contract includes two steps:

- Smart contract codes are scanned/tested for commonly known and more specific vulnerabilities using automated analysis tools.
- Manual audit of the codes for security issues. The contracts are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that was considered during the audit of the smart contract:

Serial Number	Audit Class	Audit Subclass
1	Overflow Audit	-
2	Reentrancy Attack Audit	-
3	Replay Attack Audit	-
4	Flashloan Attack Audit	-
5	Race Conditions Audit	Reordering Attack Audit
6	Permission Vulnerability Audit	Access Control Audit
		Excessive Authority Audit
7	Security Design Audit	External Module Safe Use Audit
		Compiler Version Security Audit
		Hard-coded Address Security Audit

Serial Number	Audit Class	Audit Subclass
		Fallback Function Safe Use Audit
		Show Coding Security Audit
		Function Return Value Security Audit
		External Call Function Security Audit

Serial Number	Audit Class	Audit Subclass
7	Security Design Audit	Block data Dependence Security Audit
		tx.origin Authentication Security Audit
8	Denial of Service Audit	-
9	Gas Optimization Audit	-
10	Design Logic Audit	-
11	Variable Coverage Vulnerability Audit	-
12	"False Top-up" Vulnerability Audit	-
13	Scoping and Declarations Audit	-
14	Malicious Event Log Audit	-
15	Arithmetic Accuracy Deviation Audit	-
16	Uninitialized Storage Pointer Audit	-

3 Project Overview

3.1 Project Introduction

This is a core cross-chain bridge module built around EdgexBridge and CctpExtension. The former is responsible for user asset deposits, withdrawal queuing, signature verification, release execution, and rate limiting, while the latter handles USDC conversion, Circle CCTP burn/mint cross-chain settlement, and liquidity coordination across bridge endpoints.

The overall design is essentially a highly governed, upgradeable cross-chain fund management system that relies on off-chain signatures and centralized operational processes.

3.2 Vulnerability Information

The following is the status of the vulnerabilities found in this audit:

NO	Title	Category	Level	Status
N1	Potential Risk of Deposit Not Restricting Token Types	Design Logic Audit	Medium	Acknowledged
N2	<code>deposit</code> Records the Caller Instead of the Actual Depositor in CCTP Deposit Flows	Others	Suggestion	Acknowledged
N3	Potential Risk of Using Permit for WETH Deposits	Design Logic Audit	High	Acknowledged
N4	<code>clientWithdrawId</code> Lacks Global On-Chain Uniqueness Enforcement	Design Logic Audit	Suggestion	Acknowledged
N5	<code>setRateLimitConfig</code> Directly Overwrites Configuration, Causing Potential Rate Limit State Inconsistency	Design Logic Audit	Low	Fixed

NO	Title	Category	Level	Status
N6	Does Not Check for Remaining Token Balances Before Switching Custodial Wallets	Design Logic Audit	Low	Acknowledged
N7	Excess Withdrawal Requests Are Not Counted Toward <code>inFlight</code>	Others	Information	Acknowledged
N8	Fixed 21,000 Gas Limit for Native Token Transfers May Prevent Withdrawals from Being Executed	Denial of Service Vulnerability	Suggestion	Acknowledged
N9	Potential Risk of <code>hubDomain</code> Configuration	Design Logic Audit	Suggestion	Acknowledged
N10	Old TokenMessenger Allowance Is Not Revoked When Updating <code>tokenMessenger</code>	Design Logic Audit	Low	Fixed
N11	<code>_swapToUsdc</code> Does Not Verify That the Input Token Is Fully Consumed	Design Logic Audit	Suggestion	Acknowledged
N12	<code>_validateAndDecode</code> Has Insufficient Trust Boundary Validation for the CCTP Receive Path	Design Logic Audit	Suggestion	Acknowledged
N13	<code>renounceOwnership</code> Not Disabled, Creating a Potential Risk of Permanent Protocol Loss of Control	Authority Control Vulnerability Audit	Suggestion	Fixed
N14	Overly Centralized Privilege Model in the	Authority Control Vulnerability Audit	Suggestion	Acknowledged

NO	Title	Category	Level	Status
	Bridge Protocol			
N15	Redundant receive Function Implementation	Others	Suggestion	Acknowledged

4 Code Overview

4.1 Contracts Description

Audit Version:

edgex-contracts-dc104d97-audit.zip:

8335ab84b9befbc2e1e7077fa131905a5707850d5b67379e5d4bb3e5e63c3c4b

Fixed Version:

edgex-contracts-feat-withdrawal-rate-limit-v2.zip:

cdf8cdfc9c2dfcc62d4f83ad74e82c282b76fabcf1dc72e80c904f31c868881a

Audit Scope:

```
./src/new
├── CctpExtension.sol
├── CctpExtensionStorage.sol
├── EdgexBridge.sol
├── EdgexBridgeStorage.sol
└── interfaces
    ├── ICctpExtension.sol
    └── IEdgexBridge.sol
```

The main network address of the contract is as follows:

The code was not deployed to the mainnet.

4.2 Visibility Description

The SlowMist Security team analyzed the visibility of major contracts during the audit, the result as follows:

CctpExtension			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
initialize	External	Can Modify State	initializer
depositWithSwap	External	Can Modify State	nonReentrant
depositCrossChain	External	Can Modify State	nonReentrant
depositLocalWithSwap	External	Can Modify State	nonReentrant
rebalanceTo	External	Can Modify State	nonReentrant
withdrawTo	External	Can Modify State	nonReentrant
relayDeposit	External	Can Modify State	nonReentrant
setUsdc	External	Can Modify State	onlyOwner
setTokenMessenger	External	Can Modify State	onlyOwner
setMessageTransmitter	External	Can Modify State	onlyOwner
setCircleAdapter	External	Can Modify State	onlyOwner
setBridge	External	Can Modify State	onlyOwner
setHubDomain	External	Can Modify State	onlyOwner
setDestConfig	External	Can Modify State	onlyOwner
disableDest	External	Can Modify State	onlyOwner
emergencyWithdraw	External	Can Modify State	onlyOwner nonReentrant

CctpExtension			
_authorizeUpgrade	Internal	Can Modify State	onlyOwner
_swapToUsdc	Private	Can Modify State	-
_burnToHub	Private	Can Modify State	-
_validateAndDecode	Private	-	-
_enabledDest	Private	-	-
_ensureAllowance	Private	Can Modify State	-
_self	Private	-	-

EdgexBridge			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
initialize	External	Can Modify State	initializer
<Receive Ether>	External	Payable	-
deposit	External	Payable	nonReentrant whenNotPaused
depositToWithPermit	External	Can Modify State	nonReentrant whenNotPaused
withdraw	External	Can Modify State	nonReentrant whenNotPaused
withdrawCrossChain	External	Can Modify State	nonReentrant whenNotPaused
finalize	External	Can Modify State	nonReentrant whenNotPaused
releaseBatch	External	Can Modify State	onlyOwner nonReentrant whenNotPaused

EdgexBridge			
cancelBatch	External	Can Modify State	onlyOwner
disableRateLimit	External	Can Modify State	onlyOwner
setRateLimitConfig	External	Can Modify State	onlyOwner
resetRateLimitUsage	External	Can Modify State	onlyOwner
setBaseDelay	External	Can Modify State	onlyOwner
setPenalty	External	Can Modify State	onlyOwner
setReleaser	External	Can Modify State	onlyOwner
setSigners	External	Can Modify State	onlyOwner
pause	External	Can Modify State	onlyOwnerOrPauser
unpause	External	Can Modify State	onlyOwner
setPauser	External	Can Modify State	onlyOwner
emergencyWithdraw	External	Can Modify State	onlyOwner nonReentrant
sweepStrayUsdc	External	Can Modify State	nonReentrant
setUsdc	External	Can Modify State	onlyOwner
setMscaWallet	External	Can Modify State	onlyOwner
approveMscaUsdc	External	Can Modify State	onlyOwner

EdgexBridge			
setCctpExtension	External	Can Modify State	onlyOwner
rebalanceBurn	External	Can Modify State	onlyOwner nonReentrant whenNotPaused
currentInFlight	External	-	-
withdrawalSigHash	Public	-	-
crossChainWithdrawalSigHash	Public	-	-
finalizeSigHash	Public	-	-
isAllowedSigner	Public	-	-
_authorizeUpgrade	Internal	Can Modify State	onlyOwner
_queueWithdrawal	Private	Can Modify State	-
_queueCrossChain	Private	Can Modify State	-
_reserveDelay	Private	Can Modify State	-
_payout	Private	Can Modify State	-
_execCrossChainWithdraw	Private	Can Modify State	-
_execWithdraw	Private	Can Modify State	-
_pullErc20	Private	Can Modify State	-
_setSigners	Private	Can Modify State	-
_transferNative	Private	Can Modify State	-

EdgexBridge			
_validateWithdraw	Private	-	-
_currentInFlight	Private	-	-
_verifySigners	Private	-	-
_isUsdcInMsca	Private	-	-
_ensureDistinctSigners	Private	-	-
_isNative	Private	-	-

4.3 Vulnerability Summary

[N1] [Medium] Potential Risk of Deposit Not Restricting Token Types

Category: Design Logic Audit

Content

In the `deposit` function of `EdgexBridge`, the function is responsible for receiving user deposits of native assets or ERC20 tokens. When `token` is not the native asset, it directly calls `_pullErc20` to transfer the specified amount of ERC20 tokens. However, `_pullErc20` only executes `safeTransferFrom` and does not verify whether the actual amount received by the protocol equals `amount`. It also does not enforce any whitelist restriction on supported deposit token types.

This means the protocol implicitly assumes that all ERC20 tokens follow the standard behavior that “the transferred amount equals the received amount and balances remain static after transfer.” However, this assumption does not hold for fee-on-transfer tokens, rebasing tokens, or other assets with customized transfer or accounting logic.

If a user deposits a fee-on-transfer token, the bridge contract will record the deposit based on `amount`, while the actual received asset balance may be lower than that value. If the off-chain signing system, accounting system, or withdrawal process continues to treat `amount` as the user’s withdrawable balance, an asset deficit may occur,

potentially resulting in insufficient bridge liquidity or failed withdrawals.

This issue affects not only `deposit`, but also the `depositToWithPermit` flow that reuses `_pullErc20`.

Code location: `src/new/EdgexBridge.sol`

```
function _pullErc20(address from, address token, uint256 amount) private {
    if (amount == 0) revert InsufficientBalance(0, 1);
    address recipient = _isUsdcInMsca(token) ? mscaWallet : address(this);
    IERC20(token).safeTransferFrom(from, recipient, amount);
}
```

Solution

The preferred approach is to introduce an explicit token whitelist and only allow ERC20 assets that have been verified for compatibility. Fee-on-transfer tokens, rebasing tokens, ERC777-style hook tokens, and other non-standard assets should be excluded. If broader token support is required, the implementation should compare the receiver's balance before and after the transfer and use the actual received amount, rather than the user-provided `amount`, as the basis for accounting and event emission.

Status

Acknowledged; After discussion with the project team, the team confirmed that this is an intended design.

Unsupported token deposits will be filtered by the off-chain system, and for mistakenly deposited assets, the project team will decide on a case-by-case basis whether to assist with recovery.

[N2] [Suggestion] `deposit` Records the Caller Instead of the Actual Depositor in CCTP Deposit Flows

Category: Others

Content

In the `deposit` function of `EdgexBridge`, the function is responsible for receiving user deposits of ETH or ERC20 tokens and records the deposit event through `Deposit(msg.sender, token, amount, msg.sender)`. It should be noted that `deposit` is not only called directly by regular users, but is also invoked by `CctpExtension` through `depositLocalWithSwap` and `relayDeposit`. When deposits are initiated through these paths, `msg.sender` is

actually `CctpExtension`, rather than the final user who initiated the deposit or the address that should be credited with the deposit. As a result, the emitted `Deposit` event does not accurately represent the real depositor in CCTP-based deposit flows. Off-chain systems such as event indexers, risk monitoring systems, or accounting services that rely solely on this event may incorrectly attribute deposits to `CctpExtension` instead of the actual user, leading to inaccurate user balances, reporting, or reconciliation records.

Code location: `src/new/EdgexBridge.sol`

```
function deposit(address token, uint256 amount) external payable override
nonReentrant whenNotPaused {
    ...

    emit Deposit(msg.sender, token, amount, msg.sender);
}
```

Solution

Off-chain indexers, risk systems, and reconciliation services should not rely solely on this event to identify the deposit owner. Unless this behavior is intentional, it is recommended to introduce dedicated events for `CctpExtension` - initiated deposits that explicitly include the final user address or actual depositor address. This ensures that downstream systems can correctly attribute deposits to the intended beneficiary.

Status

Acknowledged; After discussion with the project team, the team confirmed that this is an intended design.

[N3] [High] Potential Risk of Using Permit for WETH Deposits

Category: Design Logic Audit

Content

In EdgexBridge's `depositToWithPermit`, the function is intended to first obtain authorization through EIP-2612 `permit`, then pull tokens from the `to` address and record the deposit under `to`. The current implementation first directly calls `permit` and then unconditionally executes `_pullErc20`.

This design assumes that if the `permit` call fails, the transaction will always revert. However, this assumption does not always hold. If the target token does not actually implement `permit`, but instead has a fallback function or proxy forwarding mechanism that silently accepts unknown selectors, the `permit` call may appear to succeed while not creating any new allowance.

In this case, if `to` has previously granted sufficient allowance to the bridge contract, any third party could bypass the intended signature authorization semantics and call this function to pull `to`'s tokens into the bridge without possessing a valid permit signature.

Code location: `src/new/EdgexBridge.sol`

```
function depositToWithPermit(
    address to,
    address token,
    uint256 amount,
    uint256 deadline,
    uint8 v,
    bytes32 r,
    bytes32 s
) external override nonReentrant whenNotPaused {
    ...
    IERC20Permit(token).permit(to, address(this), amount, deadline, v, r, s);
    _pullErc20(to, token, amount);

    emit Deposit(to, token, amount, to);
}
```

Solution

It is recommended to only support permit-based deposits for explicitly whitelisted tokens. Before and after calling `permit` for supported tokens, the contract should additionally verify that the token's `nonces` value has increased exactly by one, ensuring that a valid permit operation was actually executed.

Status

Acknowledged; After discussion with the project team, the team confirmed that this is an intended design.

Unsupported token deposits will be filtered by the off-chain system, and for mistakenly deposited assets, the project team will decide on a case-by-case basis whether to assist with recovery.

[N4] [Suggestion] clientWithdrawId Lacks Global On-Chain Uniqueness Enforcement

Category: Design Logic Audit

Content

In the EdgexBridge contract, `clientWithdrawId` is stored as part of a pending withdrawal request and is used to identify requests across flows such as `WithdrawalQueued`, `WithdrawalExecuted`, and `WithdrawalCancelled`. However, the protocol does not maintain any dedicated on-chain state to track whether a given `clientWithdrawId` has already been used. The current replay protection relies on whether the corresponding `operationHash` already exists. However, `operationHash` also includes other parameters such as `to`, `amount`, `token`, and `expireTime`. Therefore, the same `clientWithdrawId` can still be reused to create a different `operationHash` by changing any of these parameters, allowing it to be registered as a separate withdrawal request.

This means that `clientWithdrawId` currently relies on the correctness and uniqueness guarantees of off-chain systems rather than being enforced as a unique nonce at the protocol level. If an off-chain signing service, order management system, or reconciliation system accidentally issues duplicate requests, the same `clientWithdrawId` may be reused across multiple distinct withdrawal requests, and the contract will not reject them.

Code location: `src/new/EdgexBridgeStorage.sol`

```
struct PendingWithdrawal {
    address to;
    uint64 availableAt; // request time + baseDelay (+ penalty when over cap)
    uint8 status; // 0 = Queued, 1 = Executed, 2 = Cancelled
    address token;
    uint256 amount;
    uint256 clientWithdrawId;
}
```

Solution

Unless this behavior is intentional, `clientWithdrawId` should be upgraded from a “record-only field” into a field with explicit on-chain uniqueness enforcement. The contract should maintain a mapping of used `clientWithdrawId` values and reject any withdrawal request that attempts to reuse an existing identifier. This ensures that request uniqueness is guaranteed by the protocol itself rather than depending solely on off-chain infrastructure.

Status

Acknowledged

[N5] [Low] `setRateLimitConfig` Directly Overwrites Configuration, Causing Potential Rate Limit State Inconsistency

Category: Design Logic Audit

Content

In EdgexBridge's `setRateLimitConfig`, the function is used to update a token's rate limit configuration, including the limit window (`windowSize`), maximum amount (`maxAmount`), and enabled status (`enabled`). However, the current implementation directly overwrites `rateLimitConfig[token]` without first settling the existing `rateLimitState[token]` based on the previous configuration up to the current timestamp.

The existing `rateLimitState[token].inFlight` value is calculated through linear decay using the current `cfg.windowSize`. Therefore, if an administrator changes `windowSize` while there is still historical usage recorded, the previously accumulated `inFlight` value will immediately be reinterpreted under the new window size. This can cause discontinuous jumps in the rate limit state and break the consistency of the intended sliding-window accounting model.

Code location: `src/new/EdgexBridge.sol`

```
function setRateLimitConfig(address token, uint256 windowSize, uint256 maxAmount,
    bool enabled)
    external
```

```

    override
    onlyOwner
    {
        if (enabled && (windowSize == 0 || maxAmount == 0)) revert
RateLimitMisconfigured();
        rateLimitConfig[token] = RateLimitConfig({windowSize: windowSize, maxAmount:
maxAmount, enabled: enabled});
        emit RateLimitConfigUpdated(token, windowSize, maxAmount, enabled);
    }

```

Solution

Before updating `rateLimitConfig[token]`, the contract should first load the previous configuration and use it to call `_currentInFlight`, settling `rateLimitState[token]` up to `block.timestamp`. The resulting actual remaining `inFlight` value should then be stored as the new state baseline before applying the new configuration.

Status

Fixed

[N6] [Low] Does Not Check for Remaining Token Balances Before Switching Custodial Wallets

Category: Design Logic Audit

Content

In EdgexBridge's `setMscaWallet`, the function is used to update the protocol's current `mscaWallet` address. However, the current implementation only replaces the wallet address and does not verify whether the old `mscaWallet` has been fully drained, nor does it perform any migration process or enforce a pause requirement before switching. If the owner directly updates `mscaWallet` while the previous MSCA wallet still holds USDC, all subsequent withdrawal and rebalancing flows will operate against the new `mscaWallet` only. The USDC remaining in the old wallet will no longer be accessible through the protocol's normal operational paths. Since the native ownership of the old MSCA has already been destroyed, administrators may also be unable to use the MSCA's internal permissions to actively migrate the remaining assets.

Code location: `src/new/EdgexBridge.sol`

```
function setMscaWallet(address newMscaWallet) external override onlyOwner {
    if (newMscaWallet == address(0)) revert ZeroMscaWallet();
    address previous = mscaWallet;
    mscaWallet = newMscaWallet;
    emit MscaWalletSet(previous, newMscaWallet);
}
```

Solution

`mscaWallet` replacement should be designed as an explicit migration process rather than a direct address update.

A safer approach includes: allowing wallet migration only when the protocol is paused; requiring the old

`mscaWallet` USDC balance to be zero before switching; or performing a dedicated migration step that transfers all assets from the old wallet to the new wallet before updating the configuration.

Status

Acknowledged

[N7] [Information] Excess Withdrawal Requests Are Not Counted Toward `inFlight`

Category: Others

Content

In the EdgexBridge contract, the `_reserveDelay` function is the core rate-limiting mechanism shared by both local withdrawals and cross-chain withdrawals. It calculates `availableAt` based on `rateLimitConfig` and the current `rateLimitState`, and updates the new usage amount into `inFlight` when the request remains within the configured limit. However, when a request exceeds the configured limit, the current implementation only sets `overLimit` to `true` and applies the corresponding `penalty`. It does not record the excessive withdrawal amount into `rateLimitState[token]`.

This means that an over-limit request only affects its own maturity time and penalty calculation, but does not affect the window usage observed by subsequent withdrawal requests.

Code location: `src/new/EdgexBridge.sol`

```
function _reserveDelay(address token, uint256 amount) private returns (uint64
availableAt, bool overLimit) {
    ...
    if (inFlightNow + amount > cfg.maxAmount) {
        // Over cap → penalised, and not written into in-flight, so one
oversized
        // request can't block the window for everyone else.
        overLimit = true;
    }
    ...
}
```

Solution

N/A

Status

Acknowledged

[N8] [Suggestion] Fixed 21,000 Gas Limit for Native Token Transfers May Prevent Withdrawals from Being Executed

Category: Denial of Service Vulnerability

Content

In the EdgexBridge contract, the `_transferNative` function is responsible for executing the actual ETH transfer. During the low-level transfer call, it sets the gas limit to the fixed value `NATIVE_TRANSFER_GAS` (21,000 gas). This gas limit is only sufficient for transfers to simple externally owned accounts (EOAs). It is not reliable for contract accounts whose `receive()` or `fallback()` functions contain additional logic. For example, multisig wallets, proxy wallets, contract-based accounts with event emission, or addresses performing additional security checks may require more than 21,000 gas and will revert due to insufficient gas.

Code location: `src/new/EdgexBridge.sol`

```
uint256 internal constant NATIVE_TRANSFER_GAS = 21_000;
function _transferNative(address to, uint256 amount) private {
```

```
(bool success,) = to.call{value: amount, gas: NATIVE_TRANSFER_GAS}("");
if (!success) revert NativeTransferFailed(to, amount);
}
```

Solution

Unless this behavior is an intentional design choice, the fixed 21,000 gas limit should be removed. The transfer should use a standard call pattern `call{value: amount}("")` . while retaining the existing `nonReentrant` protection and following the checks-effects-interactions pattern to mitigate reentrancy risks.

Status

Acknowledged; After discussion with the project team, the team confirmed that this is an intended design.

[N9] [Suggestion] Potential Risk of `hubDomain` Configuration

Category: Design Logic Audit

Content

In the `CctpExtension` contract's `initialize` function, the function only initializes `owner_` and leaves `hubDomain` to be configured separately through `setHubDomain` after deployment. When `hubDomain` has not yet been initialized, its default value is `0` . However, since `0` is itself a valid CCTP domain value, the protocol cannot determine whether the field has been correctly configured simply by checking whether it equals `0` . As a result, if the operations team forgets to call `setHubDomain` after deployment or upgrade, the protocol paths that depend on `_burnToHub` may continue executing with `hubDomain = 0` . This could cause USDC to be burned for an unintended destination domain, resulting in incorrect cross-chain routing and potential loss of funds.

Code location: `src/new/CctpExtension.sol`

```
function initialize(address owner_) external initializer {
    if (owner_ == address(0)) revert ZeroAddress();
    __Ownable_init(owner_);
}
```

Solution

The preferred approach is to include `hubDomain` as an initialization parameter and configure it together with `owner_` during `initialize`, with explicit deployment-time validation requirements. If post-deployment configuration is required due to CREATE2 address consistency or deployment workflow constraints, all entry points that depend on `_burnToHub` should include an explicit check ensuring that `hubDomain` has been properly configured before execution. This prevents the protocol from performing cross-chain burns with an uninitialized or unintended domain value.

Status

Acknowledged

[N10] [Low] Old TokenMessenger Allowance Is Not Revoked When Updating `tokenMessenger`

Category: Design Logic Audit

Content

In the `CctpExtension` contract's `setTokenMessenger` function, the function is used to update the `tokenMessenger` address currently used for CCTP burn operations on the chain. During normal operation, the protocol grants an unlimited USDC allowance to the current `tokenMessenger`. However, `setTokenMessenger` only overwrites the stored `tokenMessenger` address and does not revoke the allowance granted to the previous address before switching.

As a result, if the protocol previously granted an unlimited USDC allowance to the old `tokenMessenger`, that allowance will remain active in the USDC contract even after the address has been replaced. Although subsequent business logic will only use the new `tokenMessenger`, the old address could theoretically still call `transferFrom` to withdraw USDC held by `CctpExtension` if it is later misconfigured, compromised after deprecation, or affected by flaws in its permission model or upgrade mechanism.

Although this contract is designed not to permanently hold significant balances under normal circumstances, `CctpExtension` may still temporarily hold USDC during cross-chain processing windows, message execution

delays, abnormal transfer scenarios, or emergency situations. Therefore, leaving historical unlimited allowances active increases the attack surface introduced by configuration changes.

Code location:

```
function setTokenMessenger(address newTokenMessenger) external override onlyOwner
{
    if (newTokenMessenger == address(0)) revert ZeroAddress();
    tokenMessenger = newTokenMessenger;
    emit TokenMessengerSet(newTokenMessenger);
}
```

Solution

Before updating `tokenMessenger`, the contract should first read the existing `tokenMessenger` address and explicitly revoke its USDC allowance by setting it to zero. Only after the old allowance has been removed should the contract update the stored `tokenMessenger` address and configure the new authorization.

Status

Fixed

[N11] [Suggestion] `_swapToUsdc` Does Not Verify That the Input Token Is Fully Consumed

Category: Design Logic Audit

Content

In the `CctpExtension` contract, the `_swapToUsdc` function is responsible for pulling `p.amountIn` of `tokenIn` from `msg.sender`, authorizing `circleAdapter` to execute the Circle signature-based swap flow, and determining the swap output based on the increase in the contract's USDC balance. Although the current implementation grants an exact allowance for `amountIn` before execution and resets the allowance to zero afterward, it does not verify whether the adapter actually consumed the entire `p.amountIn` amount of `tokenIn`.

This means that if the adapter execution path only partially consumes the input token, or if residual tokens remain due to routing behavior, rounding differences, path design issues, or non-standard token behavior, the function may

still successfully return as long as the received USDC amount meets expectations.

From the user's perspective, the entire `p.amountIn` amount has already been transferred into the contract through `safeTransferFrom`, but any portion of `tokenIn` that was not consumed by the adapter will not be automatically returned to the user and will not be included in the subsequent bridge amount calculation. Instead, the remaining tokens will remain trapped in `CctpExtension`. This can lead to unexpected token accumulation inside the contract and creates an accounting mismatch between the user's input amount and the actual amount used for the swap.

Code location: `src/new/CctpExtension.sol`

```
function _swapToUsdc(
    ...
) private returns (uint256 usdcOut) {
    ...
    // Exact allowance, reset after -- bounds the adapter to this call's input,
    not residuals.
    IERC20(p.tokenIn).forceApprove(circleAdapter, p.amountIn);

    uint256 balanceBefore = IERC20(usdc_).balanceOf(address(this));
    ICircleAdapter(circleAdapter).execute(exec, tokenInputs, signature);
    IERC20(p.tokenIn).forceApprove(circleAdapter, 0);
    usdcOut = IERC20(usdc_).balanceOf(address(this)) - balanceBefore;

    if (usdcOut < p.minUsdcOut) revert InsufficientSwapOutput(usdcOut,
p.minUsdcOut);
}
```

Solution

Before and after executing the swap, the contract should record the `tokenIn` balance of the contract and verify that the expected amount has been fully consumed. The remaining unspent balance should either be required to be zero (or within an explicitly accepted minimal tolerance) or the transaction should revert. Alternatively, if residual input tokens are allowed by design, the remaining `tokenIn` should be explicitly returned to the original caller after execution. This ensures that no user funds are unintentionally stranded inside `CctpExtension`.

Status

Acknowledged

[N12] [Suggestion] _validateAndDecode Has Insufficient Trust Boundary Validation for the CCTP

Receive Path

Category: Design Logic Audit

Content

In the `CctpExtension` contract, the `_validateAndDecode` function is responsible for performing local pre-validation of CCTP messages submitted through `relayDeposit`. However, the current implementation does not further verify that `MessageV2._getRecipient` matches the currently configured `tokenMessenger`. This means the current validation logic implicitly assumes that as long as `messageSender` and `mintRecipient` are correct, the message must be a legitimate CCTP burn message intended for the expected `tokenMessenger`. As a result, the trust boundary is weakened from “only accept genuine CCTP burn messages processed by the configured TokenMessenger” to “accept any Circle-authenticated message whose body can be interpreted as a burn message.” Without validating the recipient field, a message that was authenticated by Circle but originally intended for another recipient context could potentially pass local validation if its payload satisfies the existing checks. This creates a mismatch between the intended CCTP message flow and the actual guarantees enforced by the contract.

Code location: `src/new/CctpExtension.sol`

```
function _validateAndDecode(bytes29 parsed) private view returns (address to) {  
    ...  
}
```

Solution

Add an explicit validation in `_validateAndDecode` requiring `MessageV2._getRecipient` to equal the currently configured `tokenMessenger`. This ensures that only messages originating from the expected CCTP

TokenMessenger flow can enter the `relayDeposit` processing path and preserves the intended trust boundary of

the receive mechanism.

Status

Acknowledged

[N13] [Suggestion] renounceOwnership Not Disabled, Creating a Potential Risk of Permanent Protocol

Loss of Control

Category: Authority Control Vulnerability Audit

Content

In the EdgexBridge and CctpExtension contracts, both core contracts inherit from `Ownable2StepUpgradeable` and bind many critical administrative capabilities to the `owner` role. Under this strong owner-dependent architecture, retaining OpenZeppelin's default `renounceOwnership` functionality introduces a significant operational risk. If the owner accidentally calls `renounceOwnership`, the protocol will immediately lose control over critical management functions, including parameter updates, upgrades, emergency operations, and recovery mechanisms. A single accidental operation could therefore place the system into an unrecoverable state where no authorized party can perform required administrative actions.

Solution

It is recommended to explicitly override `renounceOwnership` in both `EdgexBridge` and `CctpExtension` and make the function revert permanently.

Status

Fixed

[N14] [Suggestion] Overly Centralized Privilege Model in the Bridge Protocol

Category: Authority Control Vulnerability Audit

Content

In the EdgexBridge and CctpExtension contracts, both core components adopt an owner-centric administrative model. In EdgexBridge, the owner is not only responsible for upgrade authorization, but also manages signer

configuration, releaser and pauser roles, rate limit parameters, USDC/MSCA/CCTP-related configuration, and can perform highly privileged operations such as `releaseBatch`, `cancelBatch`, and `emergencyWithdraw`. Similarly, in `CctpExtension`, the owner controls upgrades, configuration of critical external dependency addresses, destination chain parameters, and emergency withdrawal functionality. As a result, protocol governance, asset recovery, order processing, and upgrade control all depend on a single privileged authority.

This design creates a clear single point of failure. Although the owner may be protected by a multisig wallet, this only reduces the likelihood of private key compromise or individual operational mistakes. If the privileged authority itself is compromised, governance fails, or the signing process is abused, an attacker or operator error could simultaneously affect protocol configuration, order execution, fund management, and upgrade paths, significantly amplifying the impact of a security incident. For bridge protocols, which inherently rely on off-chain coordination and cross-chain message processing, unclear privilege boundaries also increase audit complexity and the likelihood of operational mistakes.

Solution

It is recommended to replace the current owner-centric model with a role-based access control architecture. At a minimum, separate roles should be defined for upgrade authorization, parameter governance, pause and resume operations, order release, and emergency withdrawals. High-risk operations should additionally be protected by stronger governance mechanisms such as multisignature approval, timelocks, or dual-approval workflows. Privileges related to protocol upgrades and fund management should require the highest level of authorization, while routine operational parameters can be delegated to more restricted administrative roles. This approach limits the impact of any single privileged account compromise to a specific permission domain and substantially reduces the systemic governance risk of the bridge protocol.

Status

Acknowledged

[N15] [Suggestion] Redundant `receive` Function Implementation

Category: Others**Content**

In the `EdgexBridge` contract, a `receive` function is implemented, allowing any address to transfer native tokens directly to the contract without invoking a function. However, the current protocol logic does not rely on plain native token transfers to record or process user deposits. User deposits are handled through explicit function calls rather than unsolicited ETH transfers. As a result, the `receive` function does not serve any functional purpose under the current design and unnecessarily expands the contract's external interface. It may also lead to accidental transfers of native assets that are not associated with any protocol operation, increasing operational complexity.

Code location: `src/new/EdgexBridge.sol`

```
receive() external payable {}
```

Solution

If accepting direct native token transfers is not an intended feature of the protocol, it is recommended to remove the `receive` function entirely.

Status

Acknowledged

5 Audit Result

Audit Number	Audit Team	Audit Date	Audit Result
0X002607300001	SlowMist Security Team	2026.07.29 - 2026.07.30	Low Risk

Summary conclusion: The SlowMist security team uses a manual and the SlowMist team's analysis tool to audit the project. During the audit, we found 1 high-risk, 1 medium-risk, 3 low-risk, 9 suggestions, and 1 information item. All

the findings were fixed or acknowledged. The code was not deployed to the mainnet.

6 Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.



Official Website
www.slowmist.com



E-mail
team@slowmist.com



Twitter
[@SlowMist_Team](https://twitter.com/SlowMist_Team)



Github
<https://github.com/slowmist>